1-3 OS 的运行环境

- 1. OS 引导
  - 基本概念
    - **OS 引导**：将操作系统内核从外存加载到主存，并开始执行初始化程序的过程。
    - **最终结果**：操作系统内核被加载到内存，初始化完成后系统进入可用状态。
  - 引导流程
    - 1. **电源自检**：计算机启动时首先进行电源自检，确保硬件设备正常。
    - 2. **BIOS 激活**：主板 ROM 区中的 BIOS 程序被激活。
    - 3. **硬件初始化**：BIOS 负责初始化硬件，并进一步自检。
    - 4. **读取 MBR**：BIOS 读取硬盘 0 号扇区的主引导记录 MBR。
    - 5. **确定系统分区**：根据 MBR 中的分区表信息找到操作系统所在分区。
    - 6. **读取 PBR**：将系统分区 0 号扇区的分区引导记录 PBR 调入内存。
    - 7. **启动管理程序**：CPU 执行 PBR 中的代码，启动 OS 的启动管理程序。
    - 8. **加载 OS 内核**：启动管理程序找到 OS 内核位置，并将内核加载到主存。
    - 9. **内核初始化**：OS 内核开始执行初始化程序，完成后系统可用。
  - MBR
    - **全称**：Master Boot Record，主引导记录。
    - **位置**：硬盘 0 号扇区。
    - **内容**：磁盘引导程序和分区表信息。
    - **作用**：帮助系统确定应该从哪个分区继续启动。
  - PBR
    - **全称**：Partition Boot Record，分区引导记录。
    - **位置**：系统分区的 0 号扇区。
    - **作用**：存放分区引导代码，用于启动 OS 的启动管理程序。
    - **多系统场景**：想启动哪个系统，就加载对应系统分区的 PBR。
  - 启动管理程序
    - **来源**：由 PBR 中的代码启动。
    - **作用**：得知 OS 内核所在位置，并把 OS 内核加载到主存。
    - **后续**：OS 内核被加载后开始执行初始化程序。
- 2. OS 内核
  - 基本概念
    - **OS 内核**：操作系统中与硬件紧密相关的模块。
    - **核心职责**：管理系统各种资源。
    - **位置特点**：常驻内存，位于内核空间。
    - **相对部分**：OS 其余部分可不一定全部常驻内存。
  - 内核空间与用户空间
    - **内核空间**：操作系统内核运行和访问的地址空间。
    - **用户空间**：普通应用程序运行的地址空间。
    - **隔离目的**：防止用户程序随意访问内核和硬件资源。
    - **安全意义**：保护 OS 和其他进程不被普通程序破坏。
- 3.OS 内核的基本功能
  - 资源管理
    - **进程管理**：保证 OS 能高效、公平、安全地同时运行多个应用程序。
    - **存储器管理**：有效、安全地分配和管理内存资源。
    - **设备管理**：保证外围设备和硬件能有效、安全地与系统交互。
  - 原语
    - **定义**：操作系统最底层的基础构件。
    - **执行特点**：运行时不可被打断。
    - **时长特点**：运行时间短。
    - **调用特点**：调用频繁。
  - 中断处理
    - **作用**：处理系统调用、键盘输入、进程调度、设备驱动等重要活动。
    - **重要性**：很多 OS 活动都依赖中断机制。
    - **本质**：让 CPU 能在需要时转入内核处理特殊事件。
  - 时钟管理
    - **定义**：OS 用来处理和控制时间及时间相关函数的机制。
    - **作用**：支撑进程调度、时间片轮转、超时控制等功能。
    - **关键词**：时间片、计时、调度。
- 4. CPU 的双重工作模式
  - 两种状态
    - **用户态**：也称目态，普通用户程序运行的状态。
    - **内核态**：也称管态，操作系统内核运行的状态。
    - **区分标志**：PSW 中的 CPU 模式位。
  - 模式位
    - **模式位 = 1**：CPU 处于用户态。
    - **模式位 = 0**：CPU 处于内核态。
    - **作用**：硬件根据模式位判断当前能否执行特权指令、访问内核资源。
  - 用户态
    - **运行对象**：普通应用程序。
    - **可执行指令**：只能执行非特权指令。
    - **访问限制**：不能直接访问系统硬件和内核软件，对内存访问也限制在用户空间。
    - **设计目的**：保护系统安全。
  - 内核态
    - **运行对象**：操作系统内核代码。
    - **可执行指令**：可以执行特权指令和非特权指令。
    - **访问权限**：对内存和系统资源的访问范围基本不受限制。
    - **设计目的**：让 OS 能管理硬件和系统资源。
- 5. 特权指令与非特权指令
  - 非特权指令
    - **定义**：用户态和内核态都可以执行的普通指令。
    - **应用程序特点**：应用程序里的所有指令都应是非特权指令。
    - **限制**：不能直接控制硬件、修改关键系统状态或访问内核空间。
  - 特权指令
    - **定义**：只能在内核态执行的指令。
    - **作用**：通常涉及硬件控制、内存管理、I/O 控制、修改 CPU 状态等敏感操作。
    - **访问范围**：在内核态下，对内存访问范围基本不受限制。
  - 用户程序执行特权指令会怎样
    - **硬件反应**：不会执行该特权指令。
    - **错误信号**：硬件发出错误信号。
    - **OS 处理**：操作系统捕捉信号后，停止该进程运行并转入错误处理程序。
    - **本质**：通过硬件和 OS 共同保护系统。
- 6. 系统调用
  - 基本概念
    - **系统调用**：用户程序请求操作系统内核服务的接口。
    - **使用场景**：文件读写、进程控制、进程间通信等。
    - **根本原因**：用户程序不能直接执行特权指令，只能请求 OS 代为完成。
  - 常见系统调用
    - **文件操作**：`open()`、`read()`、`write()`、`close()`、`lseek()`、`unlink()`、`stat()`、`fstat()`。
    - **进程控制**：`fork()`、`exit()`、`execve()`、`wait()`、`waitpid()`、`kill()`。
    - **进程间通信**：`pipe()`、共享内存、信号量、消息队列等相关调用。
  - 库函数与真正系统调用
    - **用户代码中的 read()**：通常只是一个库函数。
    - **真正的系统调用**：由库函数准备参数、设置寄存器后，通过软中断或陷阱指令发起。
    - **关键误区**：用户以为自己写的 `read()` 直接引发系统调用，其实中间通常隔着库函数封装。
  - 系统调用过程
    - 1. **用户程序调用库函数**，如 `read(fd, buf, count)`。
    - 2. **库函数准备参数**，并把必要信息放入寄存器。
    - 3. **执行软中断/陷阱指令**，发起真正的系统调用。
    - 4. **CPU 从用户态陷入内核态**。
    - 5. **内核执行对应服务例程**，完成实际操作。
    - 6. **内核返回用户空间**，把结果交回用户程序。
    - 7. **执行 IRET 等返回指令**，设置模式位回到用户态。
  - 自陷/陷入
    - **不是一条具体指令**：自陷/陷入不是指某条指令本身。
    - **触发方式**：软中断指令或陷阱指令会让 CPU 陷入内核态。
    - **本质现象**：CPU 从用户态进入内核态并执行异常处理。
    - **系统调用思路**：主动制造一次“异常事件”，让 OS 内核接管并提供服务。
- 7. 用户态与内核态切换
  - 用户态 -> 内核态
    - **用户程序发起系统调用。**
    - **发生中断**，需要 OS 内核处理中断。
    - **发生异常**，需要 OS 内核处理异常。
    - **用户程序试图执行特权指令。**
  - 内核态 -> 用户态
    - **方式**：设置 PSW 中的 CPU 模式位为 1。
    - **典型指令**：中断/异常处理结束后执行返回指令，如 `IRET`。
    - **结果**：CPU 返回用户态，继续执行用户程序。
- 8. 中断和异常系统
  - 分类总览
    - **外部中断**：由 CPU 外部设备或事件引起。
    - **内部异常**：由处理器内部异常或程序执行异常引起。
    - **内部异常细分**：终止、故障、自陷。
  - 外部中断
    - **来源**：外设完成任务或发生特殊事件。
    - **作用**：请求 CPU 对外部情况进行处理。
    - **INTR**：通过 INTR 信号线发出的外中断，是可屏蔽中断。
    - **NMI**：通过 NMI 信号发出的中断，是不可屏蔽中断。
  - 终止
    - **来源**：硬件线路出现异常。
    - **特点**：一般属于严重错误，常难以恢复。
  - 故障
    - **别名**：Fault。
    - **检测时机**：在引起故障的指令启动后、执行结束前被检测到。
    - **特点**：某些故障处理后可恢复执行。
  - 自陷
    - **别名**：陷阱、陷入。
    - **特点**：与意外发生的异常不同，是预先安排的一种“异常”事件。
    - **理解**：程序主动设置一次异常，让 CPU 进入内核态执行 OS 服务。
- 9. 异常处理过程
  - 基本流程
    - 1. **异常触发**：程序执行中发生异常，CPU 自动检测并触发异常。
    - 2. **保存上下文**：保存当前执行状态，包括 PC 和寄存器值。
    - 3. **查异常向量表**：根据异常类型找到对应异常处理程序。
    - 4. **执行异常处理程序**：由内核处理异常。
    - 5. **决定处理结果**：可能修正错误继续执行，也可能终止程序。
    - 6. **恢复或终止**：可恢复则恢复状态继续执行，不可恢复则终止并报错。
  - 关键点
    - **异常通常与当前指令有关。**
    - **异常由 CPU 内部检测触发。**
    - **异常处理开始后 CPU 处于内核态。**
    - **故障类异常可能需要重新执行当前指令。**
- 10. 中断处理过程
  - 基本流程
    - 1. **触发中断**：外部设备如键盘、鼠标等发起中断。
    - 2. **CPU 检测中断**：CPU 在当前指令执行完成后检测中断。
    - 3. **暂停当前进程**：暂停当前程序执行。
    - 4. **保存上下文**：保存 PC、寄存器等现场信息。
    - 5. **查中断向量表**：找到对应中断服务程序。
    - 6. **执行中断服务程序。**
    - 7. **恢复上下文并返回**：继续执行被中断的程序。
  - 关键点
    - **中断通常来自 CPU 外部。**
    - **中断一般在当前指令结束后响应。**
    - **中断处理程序开始执行时，CPU 处于内核态。**
    - **设备驱动通常依赖中断服务程序。**
- 11. 中断 vs 异常 vs 系统调用
  - 对比表

对比项	中断	异常	系统调用
来源	CPU 外部设备或事件	CPU 内部执行异常	用户程序主动请求 OS 服务
是否主动	通常被动	多数被动，自陷主动	主动
处理者	OS 内核	OS 内核	OS 内核服务例程
是否进入内核态	是	是	是
与当前指令关系	通常无直接关系	通常与当前指令有关	由陷阱/软中断指令主动触发

  - 易混点
    - **中断/异常发生时**：事件可能发生在用户态运行过程中。
    - **中断/异常处理程序开始执行时**：CPU 已进入内核态。
    - **系统调用不是普通函数调用**：库函数只是包装，真正进入内核要靠软中断/陷阱指令。
    - **sin() 这类普通库函数**：通常不需要进入内核态。

by joshuji 由 幕布 发布